

Task 1

Initially, $x = 5$ and $y = 2$.

After $x = x + y$: $x = 7$, $y = 2$.

After $y = x * 2$: $x = 7$, $y = 14$.

After $x = y - x$: $x = 7$, $y = 14$.

Final output: 7 14.

Task 2

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
```

```
    total += numbers[i]
```

```
print(total)
```

Result : 150

The program actually calculates the last element of the array, which is 50. It was probably intended to calculate the sum of all numbers. The bug is that `total` is overwritten instead of increased. The fix is to change `total = numbers[i]` to `total += numbers[i]`. The final result is 150.

Task 3

- a) $10 \times 10 = 100$ The print statement executes 100 times.
- b) $1000 \times 1000 = 1,000,000$ The print statement executes 1,000,000 times
- c) $10 \times 1,000,000 = 10,000,000$ The print statement executes 10,000,000 times.

Task 4

Algorithm A does approximately n operations, so its complexity is $O(n)$. Algorithm B does approximately n^2 operations, so its complexity is $O(n^2)$. Algorithm B grows much faster. When $n = 1,000,000$, Algorithm A needs about 1 million operations, while Algorithm B may need about 1 trillion comparisons. Therefore, Algorithm A is much faster for large n .

Task 5

1. 2^n — exponential
2. n^2 — quadratic
3. $n \log_2 n$
4. n — linear
5. $\log_2 n$ — logarithmic
6. 1 — constant

The ranking from fastest-growing to slowest-growing is: 2^n , n^2 , $n \log_2 n$, n , $\log_2 n$, 1. The most problematic complexities for very large n are 2^n and n^2 because their execution time grows very quickly.

Task 6

1. If I do not know that the list is sorted, I use linear search. I check the elements from left to right: 3, 8, 12, 17, 24, 31, 45. Therefore, 45 is found after 7 checks. The complexity is $O(n)$.
2. If I know that the list is sorted, I can use binary search. First, I check the middle element, 24. Since 45 is greater than 24, I search only the right half of the list. Then I find 45. This takes about 2 checks for this list. The complexity is $O(\log n)$.

Binary search can be much faster for large sorted lists because it eliminates about half of the remaining elements at each step.

Task 7

I would scan all 500 million integers and keep only the current largest number. Each number needs to be examined once.

I do not need to store all 500 million numbers. I only need to store the current maximum value.

The data can also be processed in chunks. For each chunk, I can find its local maximum and then compare the local maximums to find the global maximum.

The algorithm has $O(n)$ time complexity and $O(1)$ extra space if the numbers are processed one at a time.

Task 8

I would use a dictionary (hash map) to store each letter and its count.

I would scan the sequence from left to right. For each letter, if it is already in the dictionary, I increase its count by 1. If it is not there, I add it with a count of 1.

For A B A C B A D C A B, the result is:

A $\rightarrow$ 4
B $\rightarrow$ 3
C $\rightarrow$ 2
D $\rightarrow$ 1

The data structure I would use is a dictionary (hash map).

Task 9

Given:

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$

$A \times B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

For each of the 4 elements in the result, we need 2 multiplication operations. Therefore:

$$4 \times 2 = 8$$

So, two 2×2 matrices require 8 individual multiplication operations.

Task 10

Three possible reasons are:

1. The data may not fit in the CPU cache, so the CPU has to wait for data from main memory (RAM).
2. The memory bandwidth may be limited, so data cannot be transferred to the CPU fast enough.
3. The program may access a huge amount of data or use an inefficient memory access pattern, causing many cache misses.

Therefore, the CPU can be very fast while the program is still slow because it spends a lot of time waiting for data from memory.

Task 11

When I type `./program` in the terminal, the shell interprets the command and asks the operating system to run the program.

The operating system finds the program and loads the necessary parts into memory (RAM). Then the CPU starts executing the program's instructions.

While running, the program may read or write data in memory or access files and other resources. When the program produces output, for example using `print`, the output is sent to the terminal and I can see it on the screen.

After the program finishes, the operating system cleans up the resources used by the program.

Task 12

Program A is CPU-bound, so its main bottleneck is the CPU.

Program B is memory-bound, so its bottleneck is memory (RAM).

Program C is I/O-bound because it reads a large file, so its bottleneck is mainly disk or file I/O.

Program D mostly waits for user input, so its bottleneck is user input and waiting.

All four programs can run at the same time from the user's point of view. The operating system schedules them and shares the available resources.

CPU time is determined by the number of CPU cores, the CPU demand of each program, and the operating system scheduler.

If there is only one CPU core, only one program can execute CPU instructions at a time. The operating system switches between programs using context switching. Programs that are waiting for I/O or user input do not need to continuously use the CPU.

Task 13

I do not completely agree.

If the program is completely CPU-bound, then a computer that is twice as fast could reduce the execution time from 10 hours to about 5 hours.

However, the program may also spend time waiting for memory, disk I/O, network operations, or other resources. In that case, making the CPU twice as fast will not make the whole program twice as fast.

Therefore, the actual execution time may be less than 10 hours but it is not guaranteed to be exactly 5 hours.

Task 14

There are many ways to reduce the execution time without changing the result:

1. Improve the algorithm and use a more efficient approach.
2. Remove unnecessary calculations and operations.
3. Use more efficient data structures.
4. Reduce unnecessary memory accesses.
5. Use caching to avoid repeating the same calculations.
6. Use parallel processing for independent tasks.
7. Use multiple CPU cores or threads.
8. Optimize loops and reduce the number of iterations.
9. Use faster input/output operations.
10. Use faster hardware, such as a faster CPU, more RAM, or faster storage.

Task 15

The original execution time is 59 seconds.

If A, B, D, and E become 10 times faster:

- A = 0.2 seconds
- B = 0.2 seconds
- C = 50 seconds
- D = 0.3 seconds
- E = 0.2 seconds

The new total time is:

$$0.2 + 0.2 + 50 + 0.3 + 0.2 = 50.9 \text{ seconds.}$$

The overall speedup is:

$$59 / 50.9 \approx 1.16\times.$$

Therefore, the entire program becomes only about 1.16 times faster, or about 16% faster.

The optimization effort should focus on C because it takes 50 seconds out of the original 59 seconds. C is the main bottleneck.

Task 16

I would process the 10 million marks in one main pass.

I would keep the following data:

- min — the smallest mark
- max — the largest mark
- sum — the sum of all marks
- count90 — number of marks greater than or equal to 90
- count40 — number of marks less than 40
- frequency[101] — the number of students receiving each possible mark from 0 to 100

For every mark, I would update the minimum, maximum, sum, the two counters, and its frequency.

The average is calculated as:

$$\text{average} = \text{sum} / 10,000,000$$

For the median, I would use the frequency[101] array. Since the marks can only be from 0 to 100, I do not need to sort all 10 million values. I can scan the frequency array and find the middle position.

The data needs to be examined about 10 million times. The frequency array requires only 101 integer values, plus a few variables, so the extra memory usage is $O(1)$ relative to the number of students.

The likely bottleneck is reading and accessing the 10 million marks, especially memory or I/O.

To make it faster without changing the answer, I could divide the data between multiple CPU cores. Each core could process part of the marks and calculate local results. Then the local results could be combined to obtain the final minimum, maximum, average, median, and counts.